Smart Refrigerator Manager

Agile Project and Requirements Pack

CN7021 Advanced Software Engineering | Group project | Release 2 enhancement scope

Prepared	10 November 2026
Project members named	Sabind Daryal; Harsha (surname and student ID to be confirmed)
Delivery window	Planned through 22 December 2026 submission deadline
Scope	English localization, waste history, inventory-aware menu suggestions, and account/profile essentials. Flutter QR app deferred.

This pack turns the requested enhancement into a ScrumDesk-ready backlog, acceptance criteria, delivery plan, ownership split, architecture notes, effort estimate, and evidence checklist. It is a working project record: estimates are planning assumptions, tests and measurements are not yet recorded, and missing student/group details must be completed before submission.

Project particulars

Group number, official group size, tutorial group, tutor, module leader confirmation, student IDs, repository URL, ScrumDesk board URL, and team approvals remain to be supplied. The coursework brief specifies groups of three or four. Only Sabind and Harsha were identified in the request; confirm the official roster with the tutor before relying on this two-name allocation.

1 Introduction

This document describes the next planned release of the existing Smart Refrigerator Manager web application. The product currently manages a signed-in user's food inventory using a Java 21 / Spring Boot application, Thymeleaf pages, browser JavaScript, Spring Data JPA, and a relational database. The enhancement is intended to reduce language friction, make food disposal visible, and help users choose meals based on current stock. The proposed work is a continuation of the existing project, not the future Flutter QR scanning application. [1, 2]

Purpose and scope

Release 2 covers English/Japanese localization, a user-owned waste history, inventory-aware menu recommendations, registration with email, profile preferences, and email OTP password recovery. Flutter mobile scanning, barcode/QR intake, external grocery integrations, nutrition/allergen advice, and production AI service deployment are deferred.

Audience and reading order

Developers should read Sections 2-5 first; ScrumDesk users should use Sections 6-8; testers should use Section 9; assessors should use Sections 10-12 and the evidence checklist. Requirements are uniquely identified to support traceability, consistent with requirements engineering practice. [3]

Current state and improvement rationale

The existing application separates controllers, services, repositories, entities, and static browser code. Login and registration exist, but the registration model has no email address; the interface is Japanese; deleting food leaves no audit record; and there is no menu suggestion endpoint. The extension introduces service boundaries for waste history and menu generation while retaining the existing layered architecture. This is a substantive backend and data-model enhancement, rather than a UI-only change.

2 Product description and architecture

User classes

User	Needs
Household member	Manage own stock, choose language, record waste, get suggestions, maintain profile and recover access.
Project maintainer	Review code, configure database/mail/optional AI service, monitor errors and maintain localized text.
Assessment team	Trace backlog, requirements, implementation evidence, tests, contribution and cost assumptions.

Architecture and responsibility boundaries

Browser pages call authenticated JSON endpoints. Controllers enforce request boundaries and obtain the authenticated principal. Services apply account ownership, waste recording, recommendation selection, and profile/reset rules. Repositories persist user, food and waste-event data. The recommendation module calls an OpenAI-compatible chat-completions endpoint only when AI_API_KEY is configured; otherwise it ranks a small local recipe catalogue against English/Japanese ingredient aliases. This seam isolates provider calls and keeps the default local path usable. Spring Data JPA supplies repository abstractions; Spring Security supplies authentication and password encoding. [4, 5, 6]

Browser UI	REST controllers	Application services	JPA repositories	Relational DB
i18n, profile, inventory, waste, menus	/api/profile, /api/waste, /api/menu-suggestions	UserService, WasteService, MenuSuggestionService	UserRepository, FoodRepository, WasteEventRepository	users, foods, waste_events

The browser must not treat AI-generated recipes as allergy-safe or medically appropriate. Suggestions are ideas based on available items; users remain responsible for food safety and ingredient checks.

Design principles

• **High cohesion:** WasteService owns waste history and the stock decrement/removal transaction; MenuSuggestionService owns recipe matching and the optional provider call; UserService owns registration, profile preference, and password-reset rules.

• **Low coupling:** controllers delegate domain decisions; a configuration seam selects remote AI only by explicit environment configuration; browser translations are kept in a central message catalogue.

• **Ownership:** all inventory, profile, and history queries are scoped to the authenticated user. Any cross-user access attempt should fail closed.

• **Security:** passwords and reset codes use a one-way PasswordEncoder; reset codes expire after 10 minutes and are consumed after use. SMTP credentials and AI keys must be environment secrets, not committed to source. [7, 8]

3 Functional requirements

Priorities use MoSCoW. The English text is the acceptance language for the initial release; the interface supports Japanese and English. Requirements are statements to be validated, not evidence that validation has already passed. [3]

ID / priority	Requirement and acceptance condition
LOC-01 Must	A user can select English or Japanese on login, registration, recovery, and the dashboard. All visible labels, validation messages, and generated inventory controls switch to the selected language.
LOC-02 Must	The authenticated profile stores the preferred language. On a subsequent login, the interface restores that choice. A user can still change it in-session.
WST-01 Must	A user can record a disposal event from an owned food item with quantity and reason. The event stores a food-name/category snapshot, quantity, reason, timestamp, and owner.
WST-02 Must	Recording partial waste reduces stock by the recorded quantity; recording all remaining stock removes the active item. The event and stock change succeed or roll back together.
WST-03 Must	A user can view their waste events newest-first. A user cannot view or change another account’s events.
MENU-01 Must	A signed-in user can request up to five meal ideas based on positive-quantity inventory. Suggestions display matching inventory names, a match score and estimated preparation time.
MENU-02 Should	When AI_API_KEY and provider settings are configured, the recommendation service can request suggestions from the configured OpenAI-compatible endpoint. With no key or a provider failure, the service returns locally ranked catalogue matches.
ACC-01 Must	Registration requires username, valid email and password of at least eight characters; duplicate username/email is rejected with a useful message.
ACC-02 Must	The authenticated user can update display name and preferred language. Email is shown read-only in this iteration.
ACC-03 Must	A password reset request returns a generic response whether or not the email exists. A valid six-digit code expires after 10 minutes, can be used once, and permits password reset only after validation.
ACC-04 Must	Authentication screens expose sign-in, registration and password recovery navigation, and enforce required fields and password minimum length.
SEC-01 Must	Each authenticated endpoint derives account identity from the server-side principal and checks ownership before changing inventory or returning private data.
MOB-01 Out of scope	Flutter registration/login/dashboard and QR scanning will be a later iteration and are excluded from this web release.

4 User stories and scenarios

Story	Acceptance criteria / scenario
US-01 Language	As a household member, I want English or Japanese so I can use the system comfortably. Given a language choice, when I navigate through core screens, then labels and actions appear in the selected language; the saved choice returns after login.
US-02 Waste history	As a household member, I want to record discarded food so I can see waste patterns. Given 3 units in stock, when I record 1 wasted, then stock becomes 2 and history shows one event for 1. If I record the remaining 2, the item leaves active stock and event remains.
US-03 Menu ideas	As a household member, I want meal suggestions based on my fridge so I can use food before it expires. Given matching ingredients, suggestions show only matches from my stock and rank stronger matches first. If AI provider is unavailable, local matches remain available.
US-04 Recover account	As a registered user, I want a time-limited email code so I can regain access. Request returns the same confirmation message for any email; wrong/expired code is rejected; successful reset consumes the code and requires a normal login.
US-05 Profile	As a user, I want to update my display name and language so the app reflects my preference. Invalid/blank names and unsupported languages are rejected.

5 Nonfunctional requirements and risks

Category	Requirement / measure
Security	Passwords and OTP values are never stored as plaintext. Private endpoints require an authenticated principal. Reset responses do not disclose account existence. Rate limiting and code-attempt throttling must be added before production rollout. [7, 8]
Privacy	The optional AI provider receives inventory names, quantities and categories when configured. Configuration must be explicit, documented and covered by an appropriate privacy notice. The default local matcher sends no inventory externally.
Usability	Core actions must work on phone-width screens and desktop. English and Japanese controls must remain keyboard accessible and use associated labels.
Maintainability	Controller code should remain thin; feature logic belongs in cohesive services. Provider errors should fall back to local suggestions and be logged without credentials or reset codes.
Performance	For a household inventory under 100 items, local menu suggestion should return within 2 seconds in the development environment. Measure before claiming compliance.
Reliability	Waste recording is transactional. A failed event save must not silently decrement stock.
Risks	SMTP not configured -> OTP cannot be delivered; AI endpoint/configuration absent -> local catalogue fallback; recipe catalogue coverage limited; email not verified at sign-up; low-complexity six-digit codes require throttling; existing database migration behavior must be checked; no live ScrumDesk/Git evidence is currently connected.

6 ScrumDesk-ready backlog and agile method

Use a two-week sprint cadence with a single prioritized Product Backlog and a reviewable Increment at each sprint end. The current pack is prepared for transfer into ScrumDesk; it is not a screenshot or proof of a live board. Use columns Backlog, Ready, In Progress, Review, Done, and Blocked. Each card should contain its ID, user story, acceptance criteria, owner, estimate, dependencies and evidence link. [1]

Rank	ScrumDesk card	Estimate (points)	Owner	Depends on
1	ACC-01 Registration with email and validation	5	Sabind coding; Harsha requirements review	Existing user model/schema
2	LOC-01/02 English-Japanese UI and preference	5	Sabind coding; Harsha acceptance review	Profile endpoint
3	WST-01/02 Waste event and transactional stock update	8	Sabind coding; Harsha scenario review	WasteEvent entity/repository
4	WST-03 Waste history UI and user-scoped endpoint	5	Sabind coding; Harsha acceptance evidence	WST-01
5	MENU-01 Local inventory-ranked ideas	5	Sabind coding; Harsha recommendation review	Inventory query
6	MENU-02 Optional configured AI provider + local fallback	8	Sabind coding; Harsha privacy/acceptance review	MENU-01; API secret config
7	ACC-03 Email OTP recovery	8	Sabind coding; Harsha security criteria review	Email field; SMTP config
8	ACC-02 Profile settings and language	3	Sabind coding; Harsha usability review	Profile endpoint
9	Regression/security tests and traceability evidence	8	Sabind implements tests; Harsha reviews evidence	Stories 1-8
10	Release notes, source control and report evidence	5	Sabind code evidence; Harsha report/backlog upkeep	All earlier cards

Sprint plan (provisional)

Window	Goal	Candidate cards / exit
Sprint 1: 10-23 Nov 2026	Complete account/profile foundation and language preference.	ACC-01, ACC-02, LOC-01/02. Exit: code reviewed; English/Japanese acceptance checks recorded.
Sprint 2: 24 Nov-7 Dec 2026	Create auditable waste history and useful local menu recommendations.	WST-01/02/03, MENU-01. Exit: service ownership checks and inventory scenarios recorded.
Sprint 3: 8-15 Dec 2026	Complete optional AI integration, OTP recovery, hardening and regression.	MENU-02, ACC-03, SEC-01, regression/security tests. Exit: SMTP and AI config documented; failure fallback verified.
Release/report: 16-21 Dec 2026	Freeze evidence, assess metrics, complete report, check contribution and submission package.	Traceability, estimates, measured metrics, activity/meeting records, report finalisation.

Ceremonies and working agreements

- Sprint planning: agree one Sprint Goal, capacity and selected cards. Keep work-in-progress small; split cards that cannot be completed and reviewed inside a sprint.
- Daily coordination: short check-in on progress, next step and blockers; record date, attendees/apologies, assigned actions and next meeting in minutes.
- Sprint review: demonstrate working features against acceptance criteria and collect stakeholder feedback.
- Retrospective: identify one improvement and assign an owner/date; do not treat unverified checklist items as completed.

• Definition of Done: changes compile in the supported Java 21 environment; acceptance criteria have evidence; account ownership is reviewed; errors and secrets are handled safely; relevant tests pass; translations cover the touched flow; code is reviewed; board and report evidence are updated.

Two named participants do not meet the brief's stated group size of three or four. Scrum roles/events should therefore be treated as a proposed lightweight process until the official roster is confirmed. Do not fabricate a third or fourth participant.

7 Work allocation and contribution plan

The requested responsibility boundary is explicit: Sabind performs all coding. Harsha is allocated non-coding work including requirements clarification, backlog upkeep, acceptance review, meeting minutes, risk/evidence tracking, and report coordination. Estimates below are initial allocation estimates, not verified time sheets or proof of equal contributions. The assessment brief asks for nearly equal participation and a contribution statement signed by all members; reconcile this plan with the official group and actual records before submission. [2]

Work package	Sabind Daryal	Harsha	Evidence to retain
Coding: backend, frontend, data model, integrations	Primary; all coding	No coding assigned	Commits/PRs and dated change log
Requirements and acceptance criteria	Technical clarification	Primary; review/approve criteria	Backlog history and review notes
ScrumDesk board	Update implementation states	Primary board hygiene / sprint records	Board export or screenshot with date
Testing	Implement automated tests and fixes	Execute/review acceptance scripts and record results	Test run output and signed scenarios
Assessment report	Architecture/code/testing input	Coordinate report structure, references, meeting minutes	Versioned report drafts and source list
Risks, metrics, COCOMO	Technical size/complexity data	Maintain estimates, risks, measurement log	Baseline and post-change measurement notes

Illustrative effort allocation (planning only)

Area	Sabind estimate (h)	Harsha estimate (h)	Status
Coding / implementation	46	0	Sabind coding allocation; estimate only
Requirements / backlog / review	8	18	Harsha leads review; Sabind supports technical detail
Testing / evidence	10	12	Testing remains unexecuted in this record
Report / planning / meeting records	8	18	Shared inputs; actual contributions to be logged
Total planned hours	72	48	Unequal estimates; revise with real capacity and official roster

The illustrative allocation is not a signed declaration that contributions were equal. If coursework policy requires equivalent individual contribution, adjust non-coding work and record actual activity. Preserve meeting minutes with date, attendees and apologies, next actions, next meeting time/date/venue, and member agreement/signatures.

8 Project planning, cost and estimation

A preliminary COCOMO 81 Basic organic estimate is used only as a cross-check. The planning size is 0.40 KLOC of net new enhancement code, not a measured source count. Basic organic effort is 2.4 x KLOC^1.05 person-months; schedule is 2.5 x effort^0.38 months. This gives approximately 0.93 person-month and 2.42 months. At a planning conversion of 160 hours per person-month and an assumed £20 per hour, the indicative labour budget is about 149 hours / £2,980. This estimate includes uncertainty and is not a quote or actual cost. COCOMO is sensitive to size and cost-driver assumptions; recalculate using a measured LOC/function-point estimate and local rates before final submission. [9]

Input / output	Value	Basis
Net new software size	0.40 KLOC	Planning assumption; measure by feature, exclude generated/vendor code
Effort	2.4 x 0.40^1.05 = 0.93 PM	COCOMO Basic organic mode
Schedule	2.5 x 0.93^0.38 = 2.42 months	Nominal estimate
Labour hours	0.93 x 160 = 149 h	Assumed 160 h / PM
Cost	149 x £20 = £2,980	Assumed blended hourly rate, not market data

Risk register

Risk	Likelihood / impact	Mitigation / owner
Group roster mismatch	Medium / High	Confirm official group size and students with tutor. Harsha to track; Sabind supplies technical evidence.
Mail service unavailable or misconfigured	Medium / High	Use environment-based SMTP; verify delivery in a controlled mailbox before claiming feature complete. Sabind.
OTP brute-force / reset flooding	Medium / High	Add per-email/IP throttling, attempt limits and monitoring before production release. Harsha tracks security acceptance; Sabind implements.
External AI key or service unavailable	Medium / Medium	Keep key out of source; timeout and fallback; test no-key/provider-error paths. Sabind.
Inventory data leaves service boundary	Low if disabled / High	Default local-only; disclose remote processing, minimise data, review provider terms and privacy notice before enablement. Harsha tracks; Sabind implements config.
Unverified feature behaviour	High / High	Run planned unit/integration/acceptance tests on Java 21; record evidence and defects. Both.
No version-control history currently available	High / High	Initialize/connect the official remote, work in traceable branches and preserve dated commits before final evidence freeze. Sabind; Harsha tracks.

9 Verification and software metrics

No test run or code-quality measurement is claimed in this pack. The project contains a Spring Boot test dependency and a starter test class, but the new requirements need explicit test cases and results. The checklist below defines the evidence to collect. Unit and integration testing should be run through the project’s Maven lifecycle after a Java 21 runtime and database profile are available. [10, 11]

ID	Test idea	Expected outcome / evidence
T-LOC-01	Switch language on each account screen and dashboard.	All visible strings translate; preference persists after sign-out/sign-in. Screenshot or scripted result.
T-WST-01	Record partial and full waste; inspect stock and history.	Partial count decrements; full count removes active row; event remains; rollback on forced persistence failure.
T-WST-02	User A attempts access to User B event or food ID.	Request denied and no data changes. Integration/API evidence.
T-MENU-01	Provide English and Japanese sample ingredient names.	Relevant ideas appear, matching names and percentage are returned; unrelated/empty stock returns empty state.
T-MENU-02	Run with no key, valid test provider, invalid key and timeout.	Local path works without key; provider is used only when configured; failures fall back; secrets absent from logs.
T-AUTH-01	Register duplicate username/email and weak password.	Registration rejected; valid account receives encoded password.
T-AUTH-02	Request reset for existing and unknown email; test wrong, expired, consumed code.	Generic confirmation; wrong/expired/reused code rejected; correct code resets password and normal login works.
T-PROF-01	Change display name/language; try invalid value and another account.	Valid profile persists, unsupported language rejected, ownership enforced.
T-REG-01	Run existing regression: login, food CRUD, search, filters, logout.	Previous behavior still works for each user independently.

Metrics baseline and target plan

Metric	Baseline	Target / method	Observed result
Requirement coverage	Not measured	100% of Must requirements linked to a test or accepted reason	Not measured
Test pass rate	Not measured	Report passed / executed cases; target 100% for release gate	Not measured
Endpoint response time	Not measured	Measure p50/p95 for local menu and waste history with fixed data set	Not measured
Waste history completeness	No history feature	Every accepted disposal creates one user-owned event	Not measured
Maintainability	No baseline snapshot	Record changed LOC, service responsibilities, and static analysis findings	Not measured
AI fallback	Not present	Test no-key and provider-failure cases; fallback must return local matches	Not measured

10 Continuous integration and version control

The workspace currently has no .git metadata and there is no connected GitHub/GitLab plugin or ScrumDesk session in this task. Consequently, there are no verified commits, branches, pipeline runs, board state, or board screenshots to report. Create or connect the team repository before the final submission; use feature branches named by card ID, descriptive commits, peer review for non-coding review, and a CI workflow that builds and runs tests on Java 21. Keep secrets in environment variables or the CI secret store. Maven is the project build tool; its lifecycle provides validate, compile, test, package and verify phases. [12]

Suggested board and repo evidence to capture

- ScrumDesk: board name, project goal, sprint dates, card IDs/states/owners, burndown or sprint report, review outcomes, and timestamped screenshot/export.
- Version control: repository URL and license, initial source provenance, branches, dated commits, pull requests/reviews, and attribution for each member.
- CI: workflow definition, Java 21 build/test run, pass/fail output and any deliberately recorded known failure.
- Assessment: source zip or accessible repository link, report version, test traceability, measured metrics, cost estimate assumptions, participation agreement and signed meeting records.

11 Legal, ethical and operational notes

The existing project's origin and open-source license must be verified before redistribution; no license file or upstream URL was confirmed during this work. Add proper attribution and comply with the actual license, rather than assuming the repository is permissively licensed. Email addresses are personal data. The recovery flow needs operational protections such as throttling and attempt limits, which remain follow-up work. AI mode transmits inventory metadata to a configured provider; keep local mode as the default and document the provider and data handling before enabling it. Do not present suggestions as nutritional or allergy advice. [7, 8]

12 Readiness checklist and unresolved fields

Item	State
Project topic / existing application	Smart Refrigerator Manager continuation
Features in this release	English localization; waste history; inventory-based menu recommendations; web account essentials
Flutter QR scanning app	Deferred to next update
Coding ownership	Sabind Daryal performs all coding, as requested
Harsha role	Requirements/backlog review, acceptance evidence, minutes, risks and report coordination (planned allocation)
Group member count	Only two named; official group requirement is 3-4; confirm with tutor
Student IDs / tutorial group / tutor	To be completed by team
ScrumDesk live board and screenshots	Not connected/verified; transfer backlog cards and attach board evidence
Repository URL / commits / CI	Not available in current workspace; establish and collect
Tests, runtime metrics, cost actuals	Not measured; complete before final report
Agreement of participation / meeting minutes	Must be completed, agreed and signed by all official members

References

Harvard style reference list. Web resources accessed 10 November 2026. The official brief and template are the supplied assessment materials.

[1] Schwaber, K. and Sutherland, J. (2020) *The Scrum Guide*. Available at: https://scrumguides.org/scrum-guide.html (Accessed: 10 November 2026).

[2] University of East London (2026) *CN7021 Advanced Software Engineering Coursework Brief 2026-27*. Supplied PDF, pp. 3-8.

[3] ISO/IEC/IEEE (2018) *ISO/IEC/IEEE 29148:2018 Systems and software engineering - Life cycle processes - Requirements engineering*. Available at: https://www.iso.org/standard/72089.html (Accessed: 10 November 2026).

[4] VMware, Inc. (2026) *Spring Boot 3.4 Reference Documentation*. Available at: https://docs.spring.io/spring-boot/3.4/reference/ (Accessed: 10 November 2026).

[5] Spring (2026) *Spring Data JPA*. Available at: https://spring.io/projects/spring-data-jpa (Accessed: 10 November 2026).

[6] Spring (2026) *Email Support*. Spring Framework Reference. Available at: https://docs.spring.io/spring-framework/reference/integration/email.html (Accessed: 10 November 2026).

[7] Spring (2026) *Password Storage*. Spring Security Reference. Available at: https://docs.spring.io/spring-security/reference/features/authentication/password-storage.html (Accessed: 10 November 2026).

[8] OWASP Foundation (2026) *Forgot Password Cheat Sheet*. Available at: https://cheatsheetseries.owasp.org/cheatsheets/Forgot_Password_Cheat_Sheet.html (Accessed: 10 November 2026).

[9] Boehm, B.W. (1981) *Software Engineering Economics*. Englewood Cliffs, NJ: Prentice-Hall.

[10] JUnit Team (2025) *JUnit 5 User Guide*, version 5.12.1. Available at: https://docs.junit.org/5.12.1/user-guide/ (Accessed: 10 November 2026).

[11] OWASP Foundation (2026) *Email Validation and Verification in Identity Systems Cheat Sheet*. Available at: https://cheatsheetseries.owasp.org/cheatsheets/Email_Validation_and_Verification_Cheat_Sheet.html (Accessed: 10 November 2026).

[12] Apache Software Foundation (2026) *Introduction to the Build Lifecycle*. Available at: https://maven.apache.org/guides/introduction/introduction-to-the-lifecycle.html (Accessed: 10 November 2026).

[13] Oracle (2026) *JDK 21 Documentation*. Available at: https://docs.oracle.com/en/java/javase/21/ (Accessed: 10 November 2026).

[14] Fowler, M. (2018) *Refactoring: Improving the Design of Existing Code*. 2nd edn. Boston: Addison-Wesley.

[15] University of East London (2025) *CN7021 ASWE Project Report Template*. Supplied DOCX template, 2024-25 version.

Appendix A Contributions and participation

This table records the requested work split, not verified equal contribution. Complete student IDs, official group member roster and hours using actual evidence. The coursework brief requires a group participation statement signed by every member and a contributions table.

Member	Planned responsibility	Actual hours	Student ID / signature
Sabind Daryal	All coding, architecture and technical implementation. Provide commits, implementation notes and technical input to testing/report.	To log	To complete
Harsha (surname TBC)	Requirements review, ScrumDesk backlog administration, acceptance review/evidence, meeting minutes, risk and report coordination. No coding assigned.	To log	To complete
Additional official group members	Add according to official Moodle/tutor roster; do not invent names.	To log	To complete

Agreement of Participation form (complete separately and attach)

We agree to work as a group of _____ to complete the coursework for CN7021. We understand that the grade awarded will be allocated individually as a result of group work. Each listed student confirms participation and signs below.

Student name	Student ID	Email	Signature / date
Sabind Daryal			
Harsha			
Additional member (if official)			
Tutorial / practical number		Tutor name	Agreement date

Appendix B ScrumDesk card template

Card ID / title: _____
User story: As a ___, I want ___ so that ___.
Acceptance criteria: 1) ___ 2) ___ 3) ___.
Priority / estimate: ___ / ___ points. **Owner:** ___.
Dependencies / risk: ___.
Evidence link: ___. **State and date:** ___.

Appendix C Meeting minute template

Date, time, venue/link: _____
Present: _____ **Apologies:** _____
Decisions and blockers: _____
Actions (owner and due date): _____
Next meeting time/date/venue: _____
Agreement/signatures: _____